Project 25: Egg incubator

Tanner Robison

ECE 361
Fall 2026

Assigned to: Tanner Robison **GitHub:** https://github.com/TannerRobison/ECE361

Project 25. Device type A: climate regulator. **Key** K4, **second structure** S1, **report** Q6.

This sheet is your specification for the whole term. The project assignment holds the rules that apply to everyone (individual work, AI use, the final exam). This sheet says what you build, what you hand in, and how you show that it works.

1. Your device

You build an embedded system, a.k.a. a device: a small computer inside a machine that reads sensors, makes decisions, and drives the hardware. This sheet calls it your device. Yours is an incubator for chicken eggs. It holds temperature and humidity within narrow limits, turns the eggs regularly, stops turning and raises the humidity for the last days, and supports the hatch.

Device type A, climate regulator: You hold temperature or humidity near a set point, with hysteresis so that the actuators do not chatter. Switches are disturbances and overrides. The readings decide most transitions.

2. Your hardware: iom361

Every project in this course runs on the same hardware, and that hardware is a model. **iom361** is a small C library that imitates an I/O board attached to a microcontroller: switches, LEDs, an RGB LED, and an AHT20 temperature and humidity sensor.

On a real microcontroller, a program talks to hardware through **memory-mapped registers**: fixed memory addresses where each bit means something. Write a 1 to a bit of the LED register and an LED lights up; read the temperature register and you get the sensor's latest measurement. iom361 gives you exactly that, as a block of eight 32-bit registers in your computer's memory. Your code reads and writes registers the way it would on a board, but it runs on your laptop, and your tests can control what the "hardware" sees. Roy Kravitz wrote iom361 for ECE 361; this term we use version 4.0 (r4), which fixes several problems of the earlier version and adds two functions. We cover memory-mapped I/O in week 6. Until then, use iom361 as a black box through the functions below.

The files
Download the iom361 folder from Canvas. It holds:

Table 1. The iom361 files

File	What it is
`iom361_r4.h`	the interface: the register map, the return codes, and every function, with comments
`iom361_r4.c`	the library itself
`README.md`	what iom361 models, how to use it, and what changed from earlier versions
`demo_iom361_r4.c`	a short program that exercises every register; `make` builds it
`test_iom361_r4.c`	the library's own tests; `make test` runs them

Run `make` and `make test` in that folder once, before you write any code. If both work, your compiler and iom361 work together.

Then copy `iom361_r4.h` and `iom361_r4.c` into `project/`, write `#include "iom361_r4.h"` in the files that use it, and add `iom361_r4.c` to the source files in your `Makefile`. Do not change either file. If you think iom361 has a bug, ask on Agora.

The registers

Table 2. The iom361 registers

Offset	Register	Direction	What it holds
`0x00`	`SWITCHES_REG`	read	one bit per switch, bit 0 first; 1 means on
`0x04`	`LEDS_REG`	read, write	one bit per LED, bit 0 first; 1 means lit
`0x08`	`RGB_LED_REG`	read, write	bit 31 enable; bits 23 to 16 red, 15 to 8 green, 7 to 0 blue duty cycle
`0x0C`	`TEMP_REG`	read	a 20-bit raw value ST; temperature in °C = ST / 2^20 × 200 − 50
`0x10`	`HUMID_REG`	read	a 20-bit raw value SRH; humidity in %RH = SRH / 2^20 × 100
`0x14` to `0x1C`	reserved	read, write	not used in the project

The functions you use

Table 3. The iom361 functions

Function	What it does
`iom361_initialize(num_switches, num_leds, &rc)`	sets up the model and returns the base address of the registers. Call it once, at the start of `main`. Afterward the switches and LEDs are 0, the RGB LED is off, and the sensor reads 23.5 °C and 75 %RH.
`iom361_readReg(base, offset, &rc)`	reads one register. Use the names in the table above for `offset`.
`iom361_writeReg(base, offset, value, &rc)`	writes one register and returns what it holds afterward. Writes to the read-only registers are ignored.
`build_rgb_reg(enable, red, green, blue)`	builds a value for `RGB_LED_REG` from an enable bit and three duty cycles, 0 to 255
`iom361_set_display(on)`	`false` stops the model from printing the LEDs (see below)

Every function that takes `&rc` sets it: 0 for success, 1 for a wrong base address or bad arguments, 2 for an offset out of range, 3 for an offset that is not a multiple of 4, and 5 if `iom361_initialize` has not been called. Check it: a register function that fails returns `0xDEADBEEF`, which looks like a perfectly good reading.

The functions that stand in for the world

A real device gets its temperature from the air and its switch positions from a person. In the model, your code sets them:

- `_iom361_set_sensor1(temp, humidity)` sets the next reading, in °C and %RH.
- `_iom361_set_switches(value)` sets the switch register.

Only the scenario replay in `main.c` and your tests call these two. `sensor.c` and `fsm.c` never do: on a real board they would not exist.

An example

This program sets up the model for your egg incubator, plays the world for a moment, reads the sensor and a switch, and lights an LED and the RGB LED. It compiles with `gcc -std=c11 -Wall -Wextra example.c iom361_r4.c`.

```c
#include <stdio.h>
#include "iom361_r4.h"

int main(void) {
    int rc;
    uint32_t *base = iom361_initialize(3, 3, &rc);      /* 3 switches, 3 LEDs */
    if (base == NULL) {
        fprintf(stderr, "iom361_initialize failed: %d\n", rc);
        return 1;
    }

    _iom361_set_sensor1(37.5f, 52.0f);                  /* the "world": 37.5 C, 52 %RH */
    _iom361_set_switches(0x1);                          /* the "world": EGGS_IN on */

    uint32_t raw = iom361_readReg(base, TEMP_REG, &rc);
    double temp = raw / 1048576.0 * 200.0 - 50.0;       /* AHT20 formula */
    uint32_t sw = iom361_readReg(base, SWITCHES_REG, &rc);

    if (sw & 0x1u)                                      /* EGGS_IN is switch 0 */
        iom361_writeReg(base, LEDS_REG, 0x1u, &rc);     /* HEATER is LED 0 */
    iom361_writeReg(base, RGB_LED_REG, build_rgb_reg(1, 255, 0, 0), &rc); /* red */

    printf("raw 0x%05X = %.4f C, switches 0x%X\n", (unsigned) raw, temp, (unsigned) sw);
    return 0;
}
```

It prints the LEDs (__o: LED 0 on), the RGB LED, and then `raw 0x70000 = 37.5000 C, switches 0x1`. In your project this code is split up: the conversion goes into `sensor.c`, the decisions into `fsm.c`, and the calls to `_iom361_set_sensor1` and `_iom361_set_switches` into the scenario replay and your tests.

Three things to know

1. **The model prints.** Every write to `LEDS_REG` or `RGB_LED_REG` prints a line to standard output, the way a board would light up. When your program replays a scenario, it calls `iom361_set_display(false)` first, so that its trace is not mixed with the LED display.
2. **Readings are close, not exact.** The registers hold 20-bit integers, so a value comes back within 0.0001 of what was set, slightly above or slightly below: set 91.0 °C and the register gives back 90.99998 °C, which is below a 91.0 °C threshold. `sensor.c` rounds every reading to 0.1 °C and 0.1 %RH, so that a scenario that sets 91.0 reads 91.0.
3. **There is no clock.** Your program keeps time itself (see Time, below).

3. Behavior

States

The states are given. The events and the transition table are yours.

Table 4. Your states

State	Meaning
`IDLE`	no eggs (initial state)
`WARMUP`	heating to incubation temperature
`INCUBATE`	holding temperature and humidity
`TURN`	turner motor running
`LOCKDOWN`	no turning, higher humidity
`HATCH`	chicks hatching

Inputs and outputs

`iom361_initialize(3, 3, &rc)`: 3 switches, 3 LEDs. Switch k and LED k are bit k of their registers. A **level** switch means something for as long as it is on. A **press** is an event: each change of its bit from 0 to 1 counts once.

Table 5. Switches (inputs)

Bit	Switch	Kind	Meaning
0	`EGGS_IN`	press	Eggs loaded
1	`LOCKDOWN_DAY`	press	Day 18 reached
2	`HATCH_SEEN`	press	First chick hatched

Table 6. LEDs (outputs)

Bit	LED	On means
0	`HEATER`	heater on
1	`HUMIDIFIER`	humidifier on
2	`TURNER`	turner motor on

The RGB LED shows the state: one distinct color per state, listed in `SPEC.md`.

Operating rules

1. Hold 37.5 °C in every state but IDLE: heater on below 37.2 °C, off at 37.8 °C. WARMUP ends at 37.2 °C.
2. INCUBATE holds 50 to 55 %RH; LOCKDOWN and HATCH hold 65 to 70 %RH. The humidifier turns on below the band and off at its top.
3. In INCUBATE, TURN runs every 240 ticks for 6 ticks. (A real incubator turns every few hours; the model runs fast.)

Where a rule is silent, you decide. Examples you will meet: LOCKDOWN_DAY during TURN; the temperature dropping during TURN; HATCH_SEEN before LOCKDOWN_DAY. Write each decision in `SPEC.md` and test it.

4. What you build

Everything lives in `project/` in your repository.

Time

Your device keeps time the way a Unix computer does: as a `time_t`, the number of seconds since the Unix epoch, 1 January 1970, 00:00:00 UTC. The scenario file gives the start time, and the clock advances by one second per tick. Every reading gets a timestamp: the start time plus the tick.

- Store timestamps as `time_t`, and use `<time.h>`: `gmtime()` to break a timestamp into date and time, and `strftime()` to print it as `2026-10-06 08:00:46`.
- All times are UTC, so a trace is the same on every computer, whatever its time zone.
- Durations in the rules (ticks) are differences between timestamps, in seconds.
- In `SPEC.md`, answer: what would happen to your device on 19 January 2038 if `time_t` were 32 bits?

sensor.h, sensor.c

- `reading_t`: the timestamp (`time_t`), the temperature in °C, and the humidity in %RH, as floats.
- A function that reads the temperature and humidity registers and converts them with the formulas in the register table above, rounded to 0.1 °C and 0.1 %RH. Return the reading through a pointer and report failure through the return value.
- No `printf` in this module.

fsm.h, fsm.c

- An `enum` for the 6 states and an `enum` for your events.
- A transition table: an array, indexed by state and event, that gives the next state and the action. Not a chain of `if` statements.
- One function that takes the current state and an event and returns the next state.

- The code that turns a reading and the switch register into events lives here or in `main.c`. Where it lives is your decision; defend it in `SPEC.md`.

store.h, store.c: binary search tree, key K4

- `record_t`: the reading, the state at that moment, and the status word.
- An opaque type: `store.h` declares `typedef struct store store_t;`, and only `store.c` knows what is inside.
- Ordered by relative humidity, quantized to 1 %: 45.2 and 45.7 %RH have the same key. Write the comparison as one function, `record_compare`, and test it on its own.
- Duplicate keys are common. Decide what your tree does with them (a count, a list in each node, or a tie-break on the timestamp) and defend it in `SPEC.md`.
- At least: create, insert, find by key, an in-order walk with a callback, count, height, and destroy.
- Start early: implement `store.h` first as a sorted array or linked list, and replace the inside with your tree when we cover trees in week 8. If the interface is right, nothing outside `store.c` changes.

aux.h, aux.c: stack, S1

- Event history, most recent on top: every turn and phase change with its tick. Starts with room for 8 events and doubles when full. main prints the last 10 events on exit.
- Opaque type. At least: create, push, pop, peek, count, is_empty, and destroy.
- A dynamic array that doubles when it is full, as in the stack homework in week 7.

report.h, report.c: first and last occurrence of a condition, Q6

- The first and the last tick at which the temperature was outside 37.0 to 38.0 °C, and the total number of such readings.
- If the condition never held, report nothing and say so.
- It reads the store through `store.h` only.

Status word

One `uint32_t`, packed and unpacked with your HW1 `get_field` and `set_field`:

- the state (3 bits)
- `HEATER` (1 bit)
- `HUMIDIFIER` (1 bit)
- `TURNER` (1 bit)
- turn due (1 bit)

You choose the bit positions. Unused bits are 0.

main.c: the program

`main` initializes iom361, then loops: advance the clock by one second, read the sensor, read the switches, run the state machine, write the LEDs and the RGB LED, store a record, and update your stack. It has two modes:

- `./incubator scenario.txt` replays a scenario file (below) and prints one trace line per tick.
- `./incubator scenario.txt --report` replays it and prints only the report.

Scenario files

A scenario is a plain text file. The first line that is not a comment gives the start time as a Unix timestamp. Each line after it gives a tick (seconds since the start), a temperature, a humidity, and the switch register in hexadecimal. Lines starting with # are comments. Between lines, the values hold. For example:

```
start 1791273600                # 2026-10-06 08:00:00 UTC
# tick  temp   rh    switches
0       37.5   52    0x0
5       37.5   52    0x1      # EGGS_IN pressed
6       37.5   52    0x0      # and released
60      38.5   52    0x0
```

`main` feeds these values to iom361 through `_iom361_set_sensor1()` and `_iom361_set_switches()`. This format is the same for every project.

Trace output
One line per tick, so that a test can compare it with a file of expected output. For example:

```
2026-10-06 08:00:46 state=WARMUP temp=37.5 rh=52.0 leds=0x1 status=0x00000011
```

The status value depends on your bit layout.

5. Deliverables

At the freeze, **Sun Dec 6, 11:59pm**, `project/` contains:

Table 7. Deliverables

File or folder	What it holds
`sensor.[ch]`, `fsm.[ch]`, `store.[ch]`, `aux.[ch]`, `report.[ch]`, `main.c`	the code
`iom361_r4.h`, `iom361_r4.c`	the hardware model, unchanged
`Makefile`	`make` builds `incubator`; `make test` builds and runs every test; `make clean` removes everything that was built
`tests/`	unit tests per module, and scenario tests
`scenarios/`	your scenario files, each with its expected trace or report
`SPEC.md`	see below
`AI_USAGE.md`	which tools you used, for what, and one thing a tool got wrong
`ASSIGNMENT.md`	this sheet

`SPEC.md` contains:

- your events, and the transition table as a table: state, event, next state, action
- a drawing of the state machine (ASCII, or an image in the repository)
- every decision where a rule was silent
- `record_t`, and how you quantize, and what your tree does with duplicate keys
- how your stack grows, and why
- the exact definition of your report
- the bit layout of your status word
- the RGB color of each state
- the answer to the 2038 question
- one paragraph that defends or criticizes the states you were given
- one paragraph that compares your stack with one alternative from the reading, with the cost of each operation

6. How you test it

`make test` runs everything and ends with one line: the number of tests that passed and failed. It must build with `gcc -std=c11 -Wall -Wextra` with no warnings and run clean under `-fsanitize=address,undefined`.

Unit tests, at least these:

- `record_compare`: two humidities in the same 1 % step; two in neighboring steps; a humidity exactly on a step boundary.
- The clock: the start time prints as given; 59 seconds later; one second after 23:59:59 is 00:00:00 on the next day; 28 February 2027 to 1 March.

- `store`: insert into an empty tree; insert duplicates and find every one of them; find a key that is not there; the in-order walk visits records in key order; count and height after 1, 2, and 100 inserts; destroy an empty tree.
- `aux`: pop and peek on an empty stack; push past 8 so that it grows; last in, first out after growing; destroy a stack that is not empty.
- `report`: the condition never holds; it holds once; it holds at the first and at the last reading.
- The status word: pack then unpack gives back every field; unused bits are 0.
- The sensor conversion at 0 °C, at 100 °C, and at one of the thresholds in your rules.

Scenario tests, at least these seven, each in `scenarios/` with its expected output:

1. **Normal operation:** EGGS_IN, warm-up, incubation with two turns, LOCKDOWN_DAY, HATCH_SEEN.
2. **Growth:** more events than the stack's starting room of 8, so that it has to grow; check what `main` prints from it.
3. **Thresholds:** every temperature and humidity threshold in your rules, set to exactly its value. Which side of each threshold is which is your decision; the test shows it.
4. **Rule 3:** the situation in rule 3 happens, your device responds as the rule says, and then the situation clears.
5. **Unexpected input:** every switch changes at least once in every state. Nothing crashes, and the trace matches what `SPEC.md` says.
6. **Report:** a run of at least 30 readings, replayed with `--report`, whose expected output covers the special cases of your report.
7. **Midnight:** a scenario that starts at 23:59:40 and runs across midnight; the trace shows the date change.

7. When you are done

Your project is complete when:

- `make` and `make test` pass from a fresh clone, with no warnings;
- every scenario test matches its expected output;
- `SPEC.md` answers everything listed above;
- you can explain and change any line of it without help. The final exam asks you to.

8. A plan for the term

The homework builds the pieces; this is where each one lands in your egg incubator. Commit to `project/` every week.

Table 8. A plan for the term

Week	Homework	Your egg incubator
2	Modules and makefiles	the five module stubs and a `Makefile` that links them with iom361; `ASSIGNMENT.md` and a first `SPEC.md`
3	Pointers	the interfaces in `sensor.h` and `store.h`; the status word
4	ADTs, opaque types	`reading_t`, `record_t`, and `store.h`; the store as a sorted array for now
5	State machines	`fsm.c` and the transition table; scenarios 1, 3, and 4
6	iom361 registers	`sensor.c`, the clock, and `main.c`: the scenario replay and the trace; scenarios 5 and 7
7	Stacks and queues	`aux.c`: your stack; scenario 2
8	Binary search trees	the tree inside `store.c`
9	Hash tables, testing	`report.c`; scenario 6; the remaining unit tests
10	none	finish, harden, and complete `SPEC.md`